PAGE NO.
ROLL NO.

Practical 0

* **Aim:-** Introduction to R programming

* **Theory:-** R is a powerful open-source programming language and software environment designed for statistical computing, data analysis and graphical visualization. It is widely used by data scientists, statisticians, researchers and ML professionals because of its extensive collection of built-in functions and packages. R provides an easy and efficient way to manipulate data, perform statistical analysis and create high-quality visualization.

The R programming languages supports various types of data such as numeric, character, logical, complex and integer along with data structures like vectors, matrices, arrays, lists and data frames. These features make it suitable for handling both simple and complex datasets. R also includes control structures such as if-else statements, loops and functions, allowing users to write efficient and reusable programs.

One of the major strengths of R is its Rich ecosystem of packages, including ggplot2 for data visualization, dplyr for data structures manipulations, and caret for ML. R provides excellent graphical capabilities, enabling users to create bar charts, histograms, scatter plots, box plots and other visual representations of data. These visualizations help in understanding trends, patterns and relationships within datasets.

R is platform-independent and runs on windows, linux and macOS. It is commonly used in fields such as ML, DS, finance, healthcare, business analytics, bioinformatics and academic research. Due to its flexibility, simplicity and strong community support, R has become one of the most widely used programming languages for statistical analysis and data-driven decision-making.

* **Conclusion:-** The introduction of R programming provides an understanding of the basic of R language, its features and applications. It demonstrated how R simplifies statistical analysis, data manipulation and data visualization, making it an effective tool for DS, ML and research.

ROLL NO.

Practical 1

* **Aim:-** Introduction to R:-
  * Installation of R and RStudio
  * Write a simple R script that prints "Hello, World!" and outputs basic mathematical operations.

* **Theory:-** R is a free and open-source programming language developed for statistical computing, data analysis and data visualization. It is widely used by data scientists, statisticians, researchers and ML professionals because it provides powerful tools for handling, analysing and visualizing data. R supports various data structures such as vectors, matrices, arrays, lists and data frames, making it suitable for working with different types of datasets.

To write and execute R programs, two software applications are commonly used R and RStudio. R is the programming language and execution environment, while RStudio is an Integrated Development Environment (IDE) that provides a user-friendly interface for writing, editing, debugging and running R programs. It includes features such as a script editor, console, workspace viewer and plotting window.

ROLL NO.

Basic installation steps are:-

1. Download and install R from the official CRAN website.
2. Download and install RStudio Desktop.
3. Launch RStudio, which automatically detects the installed R version.
4. Create a new R script (File → New File → R Script).
5. Write and execute a simple program such as print ("Hello, World!") and perform basic mathematical operations like addition, subtraction, multiplication and division.

* **Conclusion:-** The practical introduced the R programming environment by installing R & RStudio and executing a simple R script. It helped in understanding the basic syntax of R and performing simple mathematical operations, providing a strong foundation for further learning.

PAGE NO.
ROLL NO.

Practical 2

* Aim:- R Data Types:-
  * Create variables of different data types
  * Check the class and type of each variable and convert them to another data type

* Theory:- Data types are one of the fundamental concepts of R programming, as they define the kind of values that can be stored and manipulated in a program. R supports several built-in data types, among which the most commonly used are numeric, integer, character, logical and complex. A numeric data type stores decimal numbers, an integer stores whole numbers, a character data type stores text enclosed in quotation marks, a logical data type stores Boolean values and a complex data type stores numbers with imaginary parts.

R provides built-in functions such as class() and typeof() to identify the type of a variable. The class() functions returns the category or class of an object, while typeof() returns its internal storage type. These functions help programmers understand how data is represented and procesed within R.

R also supports data type conversion (type casting), which allows one data type to be converted into another using functions such as as.character(), as.numeric(), and as.logical(). Data type conversion is useful when importing data from external sources, performing calculations, or preparing data for statistical analysis and ML. Understanding different data types and their conversions helps in writing efficient, error-free and reliable R programs.

* Conclusion:- The practical provides an understanding of different R data types such as numeric, integer, character, logical and complex. It also demonstrated how to identify the class and type of variables and convert them into different data types using built-in functions. These concepts form the foundation for effective programming.

Education to Eternity

ROLL NO.

Practical 3

* Aim:- Working with vectors:
  * Create numeric and character vectors
  * Perform operations like indexing, slicing, and arithmetic on vectors.

* Theory:- A vector is one of the most basic and important data structures in R programming. It is one-dimensional collection of elements where all values must be of the same data type, such as numeric, character, or logical. Vectors are widely used for storing and manipulating data efficiently and form the foundation for more advanced data structures like matrices & data frames.

R provides the c() (combine) function to create vectors. A numeric vector stores numbers, while a character vector stores text value. Vectors allows multiple values to be stored in single variable, making data handling and processing simpler and more efficient.

R supports various operations on vectors. Indexing is used to access a specific element by its position using square brackets ([]). Slicing allows a range or subset of elements to be selected from a vector. R also supports vector arithmetic, where operations such as add, sub, multiplication and division are performed element-wise between vectors of the same length or between a vector and a scalar value. These operations make vector manipulation fast & efficient.

* Conclusion:- The practical helped in understanding the creation of numeric and character vectors in R and performing operations such as indexing and slicing and also arithmetic. It provides a clear understanding of vector manipulation, which is a fundamental concepts of data processing, statistical analysis and ML in R.

Education to Eternity

PAGE NO.
ROLL NO.

Practical 4

* Aim:- Create lists and access elements within them
Create matrices and perform matrix operations (addition, subtraction, multiplication).

* Theory:- ① Lists in R:- A list is one of the important data structures in R programming. It is used to store multiple values or objects together in single variable. Unlike vectors, all the elements of a list do not need to be of the same data type. A list can contain numbers, characters, logical values, vectors, matrices, or even another lists.

A list is created using list() function. Each element of a list can also be given a name, which makes it easier to access the required data.

Ex:- student <- list(name = "Rishabh",
age = 23,
marks(= c(80, 85, 90)
)

Elements of a list can be accessed using the $ operator or the [[]] operator. The $ operator is commonly used when elements have names, while [[]] can be used to access element by its position.

PAGE NO.
ROLL NO.

② Matrix in R:-

A matrix is a two-dimensional data structure in R in which data is arranged in the form of rows and columns. Generally, all elements of a matrix should be of the same data type. Matrices are commonly used for mathematical calculations, statical operations, and data analysis.

A matrix can be created using the matrix() function. The number of rows and columns can be specified using nrow & ncol.

Ex:- A <- matrix (c(1,2,3,4), nrow=2, ncol=2)
This creates a matrix containing 2 rows and 2 columns.

R provides several operators for performing mathematical operations on matrices. Matrix addition is performed using the + operator, where corresponding elements of two matrices are added. Similarly, matrix subtraction is performed and multiplication also.

* Conclusion:- Thus, we successfully created a list in R and accessed its elements using different methods. Created matrix & performed matrix arithmetic operations.

ROLL NO.

Practical 5

* Aim:- Data Frames:-
① Create a data frame with student information (name, age, grades)
② Access specific rows and columns, and summarize the data.

* Theory:- A data frame is one of most commonly used data structures in R, it is used to store data in a tabular form, similar to a table in a database or a spreadsheet. A data frame consists of rows and columns, where each column represents a variable and each row represents an observation or record.

Unlike a matrix, different columns of a data frame can contain different types of data. For example, one columns can contain names as characters, another can contain age as a number and another can contain grades as numbers.

A data frame can be created using the data.frame() function.

Ex:- students <- data.frame(
Name = c("Rishabh", "Amit", "Aman"),
Age = c(22, 21, 22,)
grades = c(85, 78, 92)
)

ROLL NO.

Here, Name, Age, Grade are the columns, while each student represent one row.

Specific columns can be accessed using the $ operator. For example, Student$Name displays the name of all students, Specific rows and columns can also be accessed using row and column indexes such as students[1,] or students[,2],

R also provides several functions to summarize a data frame. The summary() function provides basic statistical information about the numerical columns, such as minimum, maximum, mean, median and quartiles. Functions such as mean(), max() and min() can also be used to calculate individual statistics.

Data frames are widely used in R for data analysis, data manipulation, statistical calculations, and visualization.

* Conclusion:- Thus, we successfully created a data frame containing student information in R. We accessed specific rows and columns using indexing and the $ operator. We summarized the data and calculated basic statistical values such as average, maximum and minimum grades.

PAGE NO.
ROLL NO.

Practical 6

* Aim:- Control Structure :-
Write an R program using if-else and for loops to calculate the factorial of a number.

* Theory:- Control structures are used in programming to control the flow and execution of a program. They help the program make decisions and repeat a set of statements when required.

In R, common control structures include if, if-else, for, while, and repeat statements.

The if-else statement is used for decision-making. It checks whether a given condition is true or false. If the condition is true, the statements inside if are executed; otherwise, the statements inside else are executed.

The basic syntax is:-

```
if (condition) {
  (# statements)
} else {
  (# statements)
}
```

A for loop is used to repeat a set of statements a fixed number of times. It is useful when we know the range or number of repetitions required.

Syntax:-

```
for (variable in sequence) {
    # statements
}
```

The factorial of a positive integer n is the product of all positive integers from 1 to n.

For example:-

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

The factorial of zero is also define as $0! = 1$

In this program, an if-else statement is used to check whether the entered number is valid for calculating factorial. If the number is valid, a for loop is used to multiply the numbers from 1 up to given number.

* Conclusion:- Thus, we successfully implemented an R program using if-else and for loop control structures to calculate the factorial of a given number. The if-else statement was used for decision-making, while the for loop was used to perform repeated multiplication.

PAGE NO.
ROLL NO.

Practical 7

* Aim:- R Functions :-
(i) Define custom functions in R
(ii) Write a function to calculate the square of a number and returns the result.

* Theory:- A function in R is a block of code designed to perform a specific task. Functions help in making programs shorter, organized, reusable and easier to understand. Instead of writing the same set of statements repeatedly, we can define a function once and use it whenever required.

R provides many built-in functions such as sum(), mean(), max() and sqrt(). Apart from these built-in functions, R also allows the programmer to create their own functions. These are called custom or user-defined functions.

A custom function is created using the function() keyword. It can accept one or more arguments, perform some operations, and return a result.

The basic syntax is :-

```
function_name <- function(argument)
{
    # statements
    return (result)
}
```

Here function_name is the name given to function, and argument is the input provided to the function. The return() statement is used to send the calculated result back to the calling program.

In this practical, a custom function named square() is created. It accepts a number as an argument, calculates its square, and returns the result.

* Conclusion:- Thus, we successfully created a custom function in R to calculate the square of a number. The function accepts a number as an argument, performs the required calculation, and return the result. This demonstrates how user-defined functions can be used to make R programs simple, reusable, and organized.

PAGE NO.
ROLL NO.

Practical 8

* Aim:- Basic statistics with R:-
  (i) Create a dataset and calculate mean, median, mode, and standard deviation
  (ii) Visualize the data using basic plots like histograms and bar plots.

* Theory:- Statistics is the process of collecting, organizing, analyzing, and interpreting data. R provides many ~~gui~~ built-in functions that make statistical calculations easy.

A dataset is a collection of related values. In this practical, we created a dataset containing the marks of students and calculate some basic measures of central tendancy and dispersion.

(1) Mean:- The mean is the average value in a dataset. In R, the mean() function is used to calculate the mean of dataset.

(2) Median:- The median is the middle value of a dataset after arranging the values in ascending or descending order. In R, the median() function is used.

(3) Mode:- The mode is the value that occurs most frequently in a dataset. R does not provide a direct statistical mode() function for finding the most frequent value, so we can create a simple custom function to calculate.

(4) Standard Deviation:- Standard deviation measures how much the value in a dataset are spread out from the mean. A small standard deviation indicates that the values are close to the mean, while a larger standard deviation indicates greater variation. In R, the sd() function is used to calculate standard deviation.

(5) Histogram:- A histogram is a graphical representation used to show the distribution of numerical data. hist() function is used.

(6) Bar Plot:- A bar plot represents the data using rectangular bars. The height ~~to~~ of each bar represents the value of a particular category. barplot() function is used.

* Conclusion:- Thus we successfully created a dataset in R and calculated its mean, median, mode and standard deviation using statistical functions. We also visualized the dataset using a histogram and bar plot, which helped in understanding the distribution and frequency of data.

PAGE NO.
ROLL NO.

Practical 9

* Aim:- Importing and Exporting data:-
  (i) Import data from CSV and Excel files into R.
  (ii) Export a data frame into a CSV file.

* Theory:- In R programming, importing and exporting data is an important part of data analysis. Data is often stored in external files such as CSV, excel, text or databases files. R provides functions and packages that allows us to read data from these files and save the processed data back to external files.

(1) Importing CSV files:-
CSV stands for Comma-Separated Values. It is a simple file format in which data is stored in rows and columns, with values usually separated by commas.

In R, the read.csv() function is used to import a CSV file.

Ex:- data ← read.csv("students.csv")

After importing the data, functions such as head(), summary() and view() can be used to examine the data.

(2) Importing Excel Files:- Excel files generally have .xlsx or .xls extensions. R does not provide a basic-built-in function for reading Excel files, so packages such as readxl can be used.

The read_excel() function from the readxl package is used to import Excel data.

Ex:- library(readxl)
data ← read_excel("students.xlsx")

The Excel file can contain different worksheets, and a particular sheets can be specified while importing.

(3) Exporting Data to CSV:-
After performing calculations or modifications on a data frame, the resulting data can be saved to a CSV file using the write.csv() function

Ex:- write.csv(data, "output.csv", row.names = False)

The row.name = False argument prevents R from adding row numbers as an extra column in the exported file.

* Conclusion:- Thus we successfully imported data from CSV and Excel files into R and exported a data frame into a CSV file. This demonstrates how R can be used to read, process, and save data in commonly used file formats.

PAGE NO.
ROLL NO.

Practical 10

* Aim:- Basic data visualization:-
  (i) Use the ggplot2 library to create a scatter plot and line graph.
  (ii) Customize the plot with labels, titles, and colors.

* Theory:- Data visualization is the process of representing data graphically so that patterns, relationships, and trends can be easily understood. R provides several libraries for creating graphs, and ggplot2 is one of the most popular libraries used for data visualization.

ggplot2 is based on the Grammar of Graphics, which provides a systematic way to build graphs by combining different graphical components. A plot can be created by specifying the dataset, variables, geometric objects, labels, and other visual properties.

Syntax :-

```
ggplot(data, aes(x = variable1, y = variable2)) +
    geom_point()
```

Here, ggplot() specifies the dataset and variables, while aes() defines how variables are mapped to the graph. Functions such as geom_point() and geom_line() determine the type of graph:

(1) Scatter plot :- A scatter plot is used to show the relationship between two numerical variables. Each observation is represented as a point on the graph.

For example, we can plot Hours studied against marks to observe whether students who study more hours tend to obtain higher marks.

The geom_point is used to create a scatter plot.

(2) Line graph :- A line graph is mainly used to show trends or changes in values. Data points are connected using lines, making it easier to observe the increase or decrease of a variable.

The geom_line() function is used to create a line graph.

(3) Customizing the plot :-

ggplot2 provides several options for customizing graphs:

* labs() is used to add titles and axis labels.
* color is used to change the color of graphical elements.
* size can be used to change the size of points or lines.
* theme_minimal() provides a simple and clean appearance.

PAGE NO.
ROLL NO.

* Conclusion:- Thus, we successfully used the ggplot2 library in R to create a scatter plot and line graph. We also customized the graphs by adding titles, axis labels, colors, and a simple theme. These visualizations helps us understand relationships and trends in the given dataset more easily.

Education to Eternity